LYRA: A-scope processing guide

Prepared by Angelo Tarzona | 13 October 2026

Scope: Set up LYRA on Windows or macOS, understand each Python file, and process a TIFF through frame detection, calibration and echo extraction.

1. What LYRA does

LYRA (Layered-echo Yield from Radiometric Archives) processes digitized A-scope radar film. It identifies oscilloscope frames within a continuous TIFF, assigns CBD identifiers, calibrates the image's time and power axes, detects surface and basal echoes, and exports travel times, echo properties, ice thickness, quality classifications, and diagnostic figures.

Main order: `detect_frames.py` → `pick_calibration.py` → `calibrate.py` → `echoes.py`.

`run_flight.py` can run these phases for an entire flight. `lyra.py` supplies functions used by the scripts rather than serving as the first command to run.

2. Files and their purposes

File	Purpose	Run routinely?
`lyra.py`	Core LYRA library: TIFF/ID resolution, frame detection, trace extraction, main-bang and calibration utilities, peak/echo measurements, and optional navigation/validation helpers. Other scripts import it.	**No**: library, not the first step.
`detect_frames.py`	**Phase 1.** Separates complete/partial A-scope frames, reads CBDs through structural 7-segment OCR or other options, and writes a per-flight frame index and visual contact sheets.	**Yes**.
`segment_ocr.py`	Supporting 7-segment character recognizer used by `detect_frames.py --method segment`; a separate F125 training/diagnostic program is also included.	**Imported automatically**; no separate command for normal processing.
`build_digit_templates.py`	Builds F125-based digit templates for the alternative normalized cross-correlation (NCC) CBD reader and exports `digit_templates.npy`. Uses a specifically named F125 training TIFF.	**No**, unless deliberately testing NCC with the required F125 TIFF.
`pick_calibration.py`	**Phase 2.** Opens interactive frames for approximate main-bang (M), -60 dB reference (R), horizontal power-grid (Y) and vertical time-grid (X) guidance picks. Saves calibration JSON.	**Yes**.
`calibrate.py`	**Phase 3.** Refines frame-specific main-bang, -60 dB reference, and grid spacing from the image and guidance picks. Produces calibrated values and diagnostic PNGs with quality-review flags.	**Yes**.
`echoes.py`	**Phase 4.** Extracts surface and bed echoes, noise level, SNR and waveform-shape metrics. Computes two-way travel times and ice thickness; generates per-frame diagnostic plots and publication PNG/EPS graphics. Supports manual review.	**Yes**.
`run_flight.py`	Orchestrates phases 1–4 over all TIFFs in one flight, with interactive calibration and QA pauses; was also designed to call a Phase 5 validation script.	**Optional shortcut** for an entire flight.

3. Use the portable scripts and the shared Dropbox folders

The original uploaded scripts point to an older project layout, and the original `run_flight.py` contains Angelo-specific `C:\Users\...` directories.

Angelo's shared folder on his Windows PC is:

```
C:\Users\dtarzona3\GaTech Dropbox\Angelo Tarzona\LYRA\
```

On PC or Mac, **the root location will differ**. Find the local Dropbox-synced folder titled `LYRA` using Explorer or Finder; the scripts automatically locate their paths relative to that folder.

The required organization is:

```
LYRA/
├── codes/
│   ├── lyra.py
│   ├── detect_frames.py
│   ├── segment_ocr.py
│   ├── build_digit_templates.py
│   ├── pick_calibration.py
│   ├── calibrate.py
│   ├── echoes.py
│   ├── otherechoes.py
│   └── run_flight.py
├── tiff/
│   ├── 47_0004800_0004824-reel_begin_end.tiff   # Single-flight: flat is OK
│   └── ... other TIFFs from the same flight ...
│   # OR organize multiple flights as tiff/127/*.tiff, tiff/125/*.tiff
├── output/                         # Created and filled by LYRA
├── requirements.txt
└── README.md
```

Important: The portable copies support **either** the existing flat `LYRA/tiff/*.tiff` directory **when it contains one flight**, **or** a numeric per-flight directory such as `LYRA/tiff/127/*.tiff`. You do not have to move large shared TIFF files if they are already all from one flight. For multiple flights, use numeric subfolders (`127`, `125`), not `F127`. The `127/4800` shorthand tells the flat-layout scripts the intended flight. Replace these example values with the real flight/TIFF IDs. Do not combine mixed flights in the flat directory.

The normal TIFF pattern is `reel_start_end-reel_begin_end.tiff`. Some scripts can rename previously renamed TIFFs using a matching `*_rename_log*.txt`; do not rename the film files manually without checking the log and mapping. Wait for the necessary TIFFs to be fully downloaded locally.

4. Windows installation (PowerShell)

1. Install **Python 3.11 or 3.12** if needed. Check that the Python Launcher (`py`) works. Open **PowerShell** (or VS Code's PowerShell terminal).
2. Navigate to your **own** local Dropbox LYRA folder. This example works only if Dropbox is directly in your Windows home folder:

```
cd "$HOME\GaTech Dropbox\Angelo Tarzona\LYRA"
py -3 --version
```

If Dropbox is elsewhere, open `LYRA` in Explorer, copy its full path, and run `cd "<your-actual-path>"` instead.

3. Create a local virtual environment **outside Dropbox** (so Windows and macOS environments do not sync over one another). Then install requirements:

```
py -3 -m venv "$HOME\lyra-venv"
$PY = "$HOME\lyra-venv\Scripts\python.exe"
& $PY -m pip install --upgrade pip
& $PY -m pip install -r .\requirements.txt
& $PY -c "import numpy, pandas, matplotlib, PIL, PyQt6; print('LYRA packages OK')"
```

Keep using `& $PY` before commands in that PowerShell session. On a later day, set `$PY = "$HOME\lyra-venv\Scripts\python.exe"` again.

5. macOS installation (Terminal)

4. Install **Python 3.11 or 3.12** if needed and open **Terminal**. On a Mac, the Dropbox folder is often somewhere under `~/Library/CloudStorage`, not `C:\Users\...`.
5. Type `cd` (including the space), drag the actual **LYRA folder** from Finder into Terminal, and press Return. Then verify Python:

```
pwd
python3 --version
```

6. Create the macOS virtual environment outside Dropbox, activate it, and install packages:

```
python3 -m venv "$HOME/lyra-venv"
source "$HOME/lyra-venv/bin/activate"
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -c "import numpy, pandas, matplotlib, PIL, PyQt6; print('LYRA packages OK')"
```

For a later session, first go back into `LYRA`, then run `source "$HOME/lyra-venv/bin/activate"` again. GUI calibration windows require a logged-in graphical desktop. Mac machines without Segoe UI may substitute another available sans-serif font in generated figures; this does not change numerical processing.

6. First test: process **one TIFF**, step by step

The worked example `127/4800` matches an F127 file with the `0004800` identifier, such as `47_0004800_0004824-reel_begin_end.tiff`. Verify that file exists in either `tiff/` (single-flight) or `tiff/127/` (per-flight). If not, substitute one that is present. Run the commands from the **LYRA root**, not from inside `codes`.

Phase 1: detect frames and recognize CBDs

Windows:

```
& $PY .\codes\detect_frames.py 127/4800 --method segment
```

Mac:

```
python codes/detect_frames.py 127/4800 --method segment
```

Open `output/F127/phase1/F127_frame_index.csv` and the TIFF's `*_contact.png` and `ocr_diag/*_ocr_diag.png`. Confirm each complete frame is correctly bounded and carries the correct CBD. **Do not assume the sequence constraint guarantees correct CBDs.** If needed, supply independently checked frame corrections using `--override FRAME:CBD` or a verified `--cbd-start` (see Section 9).

Phase 2: collect calibration guidance clicks

Windows:

```
& $PY .\codes\pick_calibration.py 127/4800
```

Mac:

```
python codes/pick_calibration.py 127/4800
```

In the calibration window, choose a mode and click approximately on the corresponding feature:

- **M:** main bang (one vertical pick; the time-zero origin).
- **R:** reference -60 dB line (one horizontal pick).
- **X:** four major vertical time-grid lines when visible.
- **Y:** two horizontal major power-grid lines when visible.
- **U:** undo the last pick in the active mode.
- **N:** save the current frame and go to the next.
- **B:** save and return to the previous frame.
- **S:** skip a frame; **Q:** save and quit the TIFF (or advance to the next TIFF in flight mode); **Esc:** save and exit.

The clicks are **guides**, not final automated picks. Outputs appear in `output/F127/phase1/F127_cal_picks.json` and `F127_cal_derived.json`. At least one frame per TIFF should have reliable M, R and X information for flight-mode QA; more guidance can improve difficult TIFFs.

Phase 3: calibrate each frame

Windows:

```
& $PY .\codes\calibrate.py 127/4800
```

Mac:

```
python codes/calibrate.py 127/4800
```

Open `output/F127/phase3/F127_cal.csv` and the `*_cal.png` plots. Check that the red line follows the main bang, the cyan line represents the -60 dB reference, and the graticule spacing follows the actual film. Correct bad guidance picks in Phase 2 and rerun Phase 3 before measuring echoes. Some frames can be flagged for review or excluded if unrecoverable.

Phase 4: detect surface/basal echoes and derive measurements

Windows:

```
& $PY .\codes\echoes.py 127/4800
```

Mac:

```
python codes/echoes.py 127/4800
```

Open `output/F127/phase4/F127_echoes.csv` and the per-frame `*_echoes.png`, `*_waveform.png` and `*_publication.png` / `*_publication.eps` files. The CSV contains surface and bed two-way travel times, power/SNR, waveform metrics, geometry and the quality status: `good`, `weak_bed`, `no_bed` or `no_surface`.

For physically interpretable ice thickness, both surface and bed picks must be credible. Ice thickness is calculated from the difference between basal and surface two-way travel times, not from the main bang alone:

`h_ice_m = (bed_twt_us - surface_twt_us) × 168 / 2`

The detection threshold or a quality label is not an independent validation. Review questionable frames visually.

Phase 4 manual review of flagged echoes

Windows:

```
& $PY .\codes\echoes.py 127/4800 --review
```

Mac:

```
python codes/echoes.py 127/4800 --review
```

Add `--all` to inspect every frame rather than just `weak_bed`, `no_bed` or `no_surface` cases. Review-mode controls include **S** for surface selection, **B** for bed selection, click to choose the feature, lowercase/uppercase suppression commands as shown in the review window, **N/Right Arrow** for next, **P/Left Arrow** for previous and **Q** to save/quit. Overrides are saved in `output/F127/phase4/F127_echo_overrides.json`. Do not run `otherechoes.py` on the same output as a routine part of this workflow.

7. Process an entire flight with one command

After the first TIFF works and you've checked CBD/calibration results, use the orchestrator. It runs frame detection, opens the calibration picker, calibrates TIFFs, performs QA and extracts echoes. You will be asked to review results between stages.

Windows:

```
& $PY .\codes\run_flight.py 127 --method segment
```

Mac:

```
python codes/run_flight.py 127 --method segment
```

Resume at a chosen phase if its earlier outputs already exist:

Windows:

```
& $PY .\codes\run_flight.py 127 --method segment --resume-from 3 --skip-picks
```

Mac:

```
python codes/run_flight.py 127 --method segment --resume-from 3 --skip-picks
```

Warning: When resuming before Phase 4, `run_flight.py` can delete the prior consolidated `F127_echoes.csv` to force re-extraction from updated calibration. Save a backup of an important existing output before reprocessing. Do not run full-flight automation simultaneously on two computers against the same shared Dropbox `output/F127` files.

8. What the outputs mean

```
output/F127/
├── phase1/
│   ├── F127_frame_index.csv         frame boundaries + CBDs
│   ├── F127_cal_picks.json          human calibration guidance
│   ├── F127_cal_derived.json        derived guide spacing
│   ├── F127_*_contact.png          TIFF overview
│   └── ocr_diag/                   OCR checks
├── phase3/
│   ├── F127_cal.csv                 frame-specific calibration
│   ├── F127_cal_review.json         calibration QA, when generated
│   └── F127_*_cal.png               calibration QA figures
└── phase4/
    ├── F127_echoes.csv              main measurements
    ├── F127_echo_overrides.json     manual review choices, when generated
    ├── F127_*_echoes.png            film + echo diagnostics
    ├── F127_*_waveform.png          waveform view
    ├── F127_*_publication.png       manuscript-oriented figure
    └── F127_*_publication.eps       vector publication figure
```

All files are written incrementally by flight; keep your original TIFFs unchanged and preserve CSV/JSON alongside figures. These Phase 1–4 outputs do **not** automatically match external navigation or BEDMAP/RIGGS references. The optional navigation routines present inside `lyra.py` require additional reference files and do not form part of this handoff.

9. Useful fixes and optional commands

Fix incorrect CBDs (example only; the values must be independently verified):

Windows:

```
& $PY .\codes\detect_frames.py 127/4800 --method segment --override 10:0471 11:0472
```

Mac:

```
python codes/detect_frames.py 127/4800 --method segment --override 10:0471 11:0472
```

Restart calibration picks for one TIFF (numeric TIFF ID):

Windows:

```
& $PY .\codes\pick_calibration.py --flight 127 --tiff 4800
```

Mac:

```
python codes/pick_calibration.py --flight 127 --tiff 4800
```

Run echo review across all previously calibrated TIFFs in one flight:

Windows:

```
& $PY .\codes\echoes.py --flight 127 --review
```

Mac:

```
python codes/echoes.py --flight 127 --review
```

Optional OCR methods: `--method ncc` requires templates generated by `build_digit_templates.py` from its specifically named F125 TIFF. `--method manual` expects an independently supplied ASTRA combined-picks CSV in `picks/<flight>/<flight>_CombinedASTRAPicks.csv`. `--method ml` and `--method ensemble` refer to `ml_ocr.py`, which was **not** provided. **Use `--method segment` for this handoff.**

10. Troubleshooting and handoff checklist

- `No module named ...`: verify the venv Python and rerun `pip install -r requirements.txt` using that interpreter.
- `Flight directory not found`: confirm there is a `tiff/` directory with `.tiff` files, either flat (one flight) or inside numeric `tiff/127/` folders (multiple flights). Do not use `tiff/F127/`.
- `No TIFF matching ID`: check that the TIFF is locally available in the expected flat or numeric flight directory and matches `*_<seven-digit-ID>_*-reel_begin_end.tiff`.
- `Frame index not found`: run `detect_frames.py` first before calibration.
- `No cal CSV`: run `calibrate.py` before `echoes.py`.
- Calibration window does not appear: check that `PyQt6` is installed and run locally in an ordinary desktop session, not a headless shell.
- OCR label visibly wrong: correct the frame index before proceeding; do not silently shift CBD identifiers to make results agree with another workflow.
- PNGs or EPS look wrong: check numerical outputs separately. macOS may use a font fallback if Segoe UI is not installed.
- Before sharing results, verify: **(a)** flight/TIFF identity, **(b)** complete vs partial frame flags, **(c)** CBD sequence, **(d)** main-bang and graticule picks, **(e)** plausible surface/basal echoes, **(f)** missing and weak echoes, and **(g)** that shared CSVs were not overwritten by a second run.